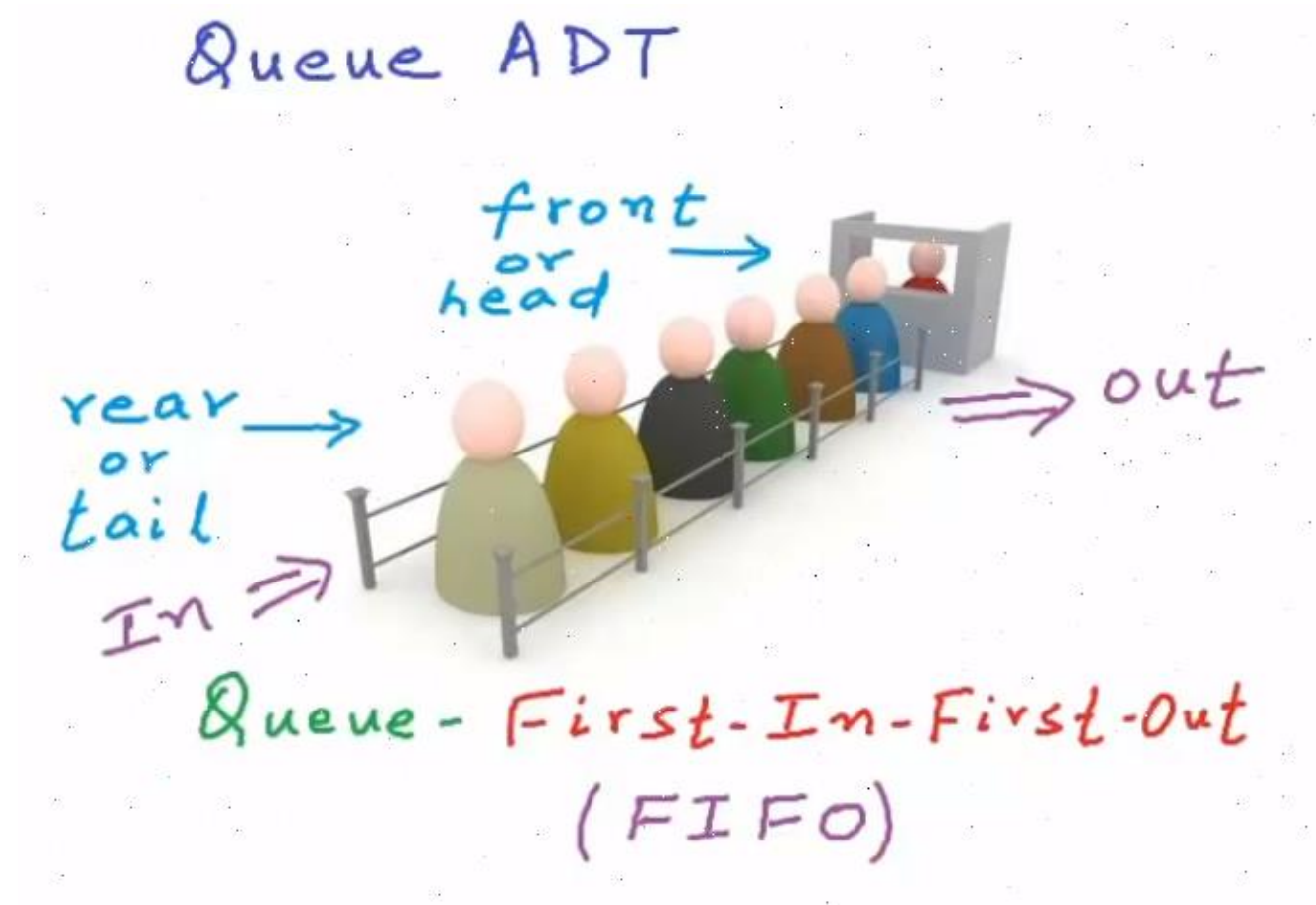


Queue

Queue



Queue Introduction

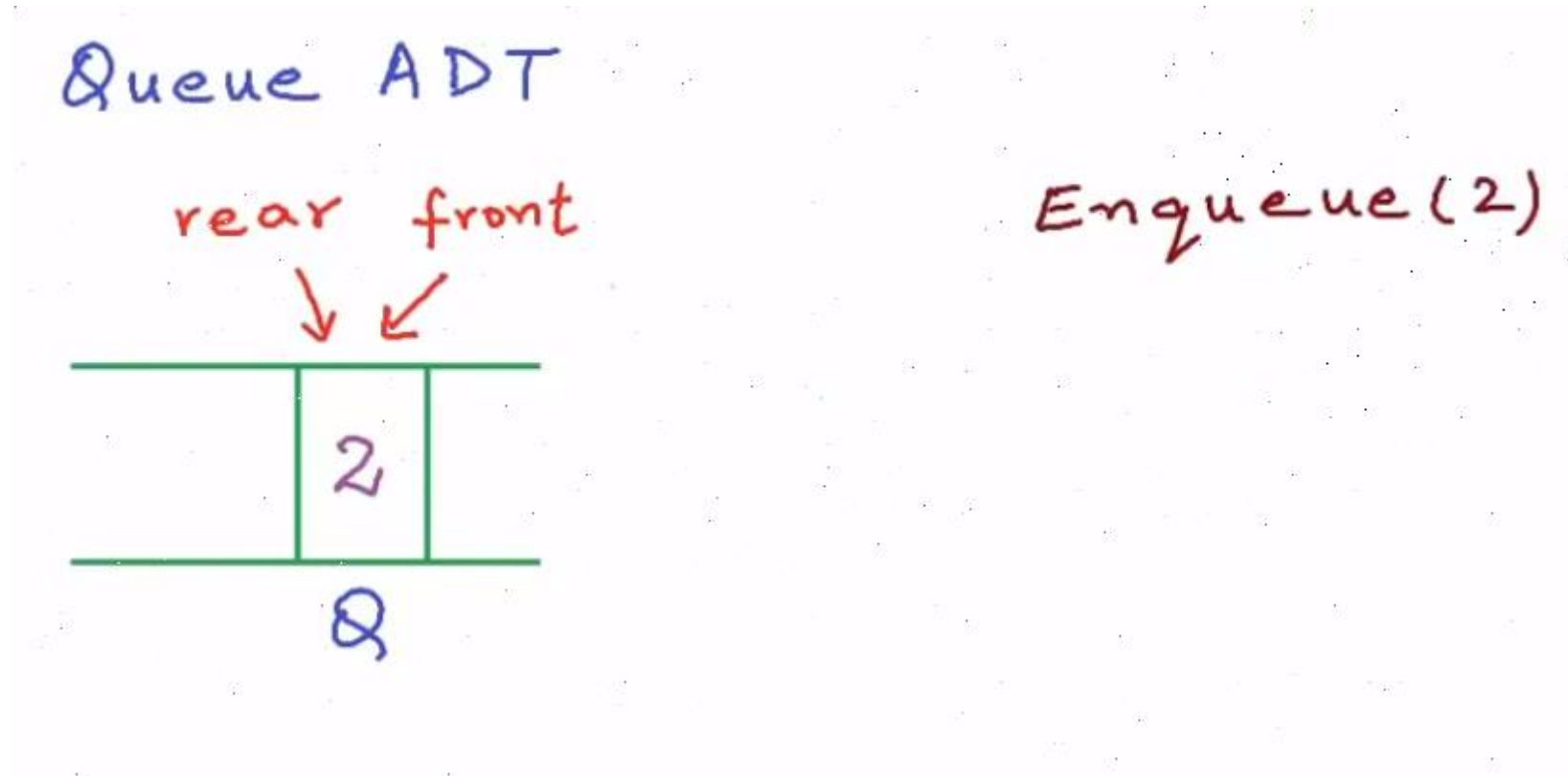
Queue ADT

A list or collection with the restriction that insertion can be performed at one end (rear) and deletion can be performed at other end (front).

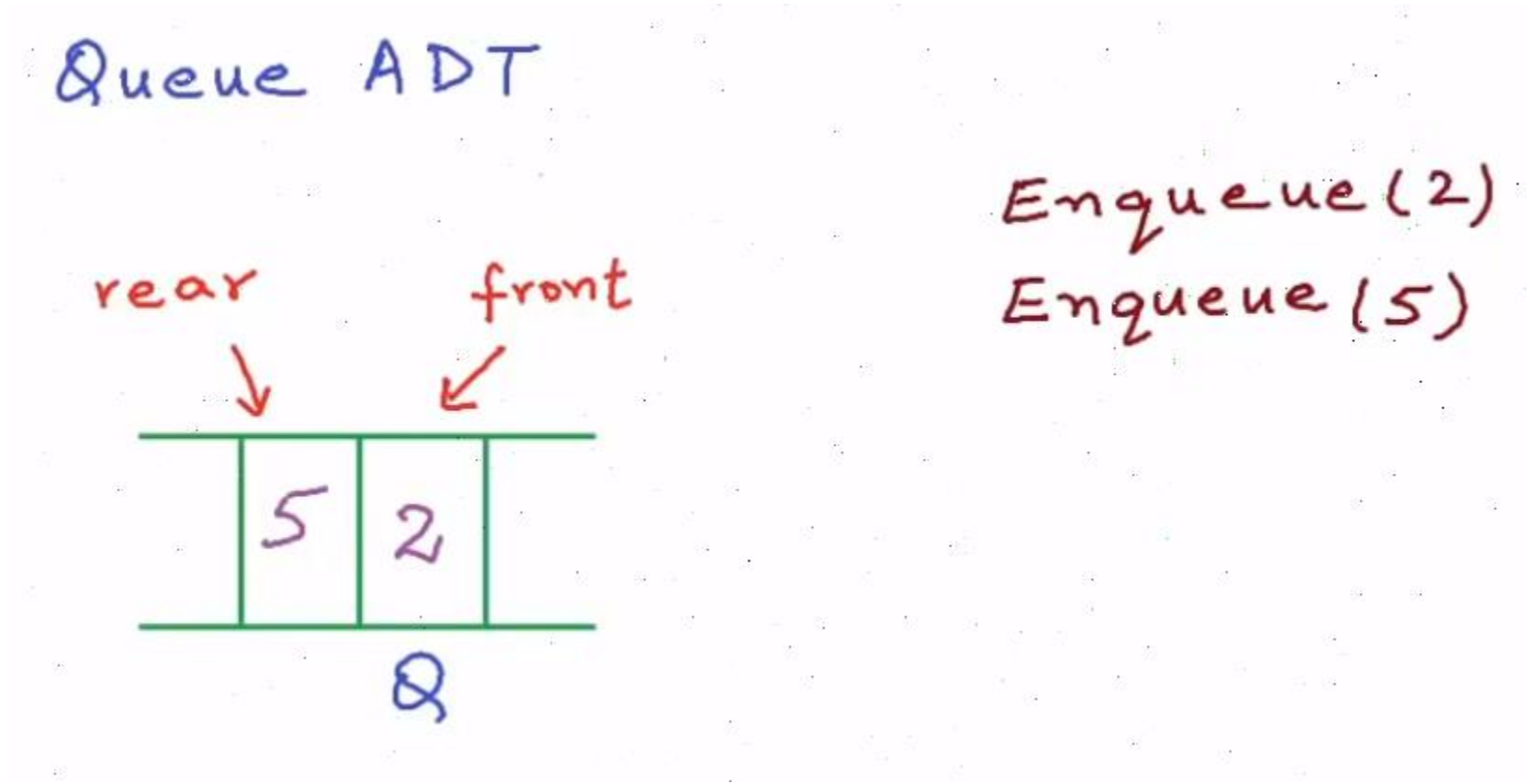
Operations

- (1) EnQueue(x)
- (2) Dequeue()
- (3) front()
- (4) IsEmpty()

Queue Operations

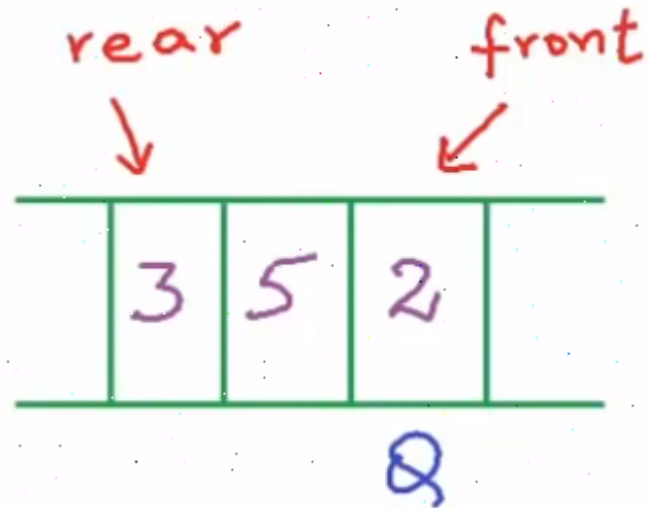


Queue Operations



Queue Operations

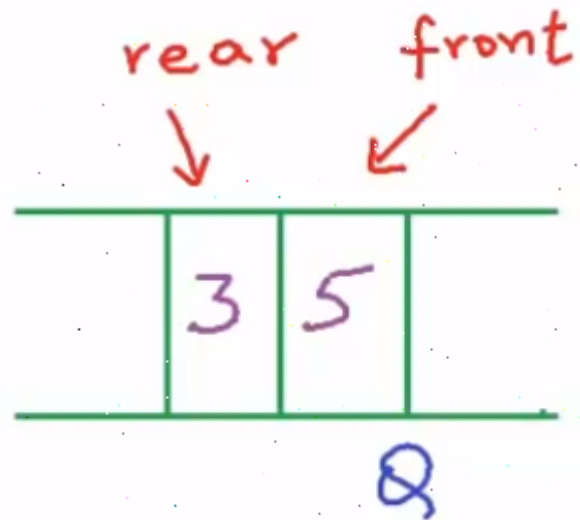
Queue ADT



Enqueue(2)
Enqueue(5)
Enqueue(3)

Queue Operations

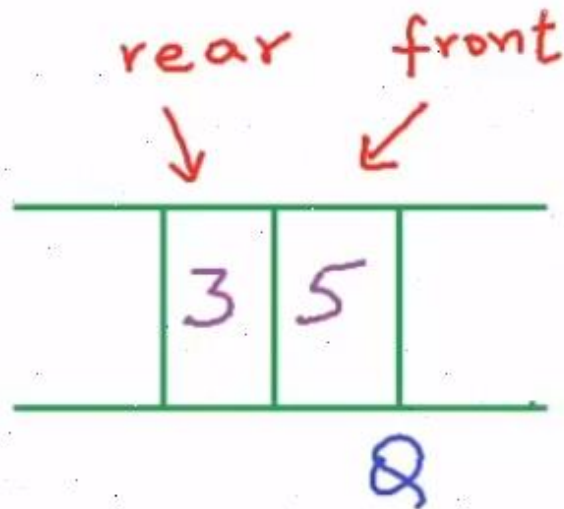
Queue ADT



Enqueue(2)
Enqueue(5)
Enqueue(3)
Dequeue()

Queue Operations

Queue ADT



Enqueue(2)

Enqueue(5)

Enqueue(3)

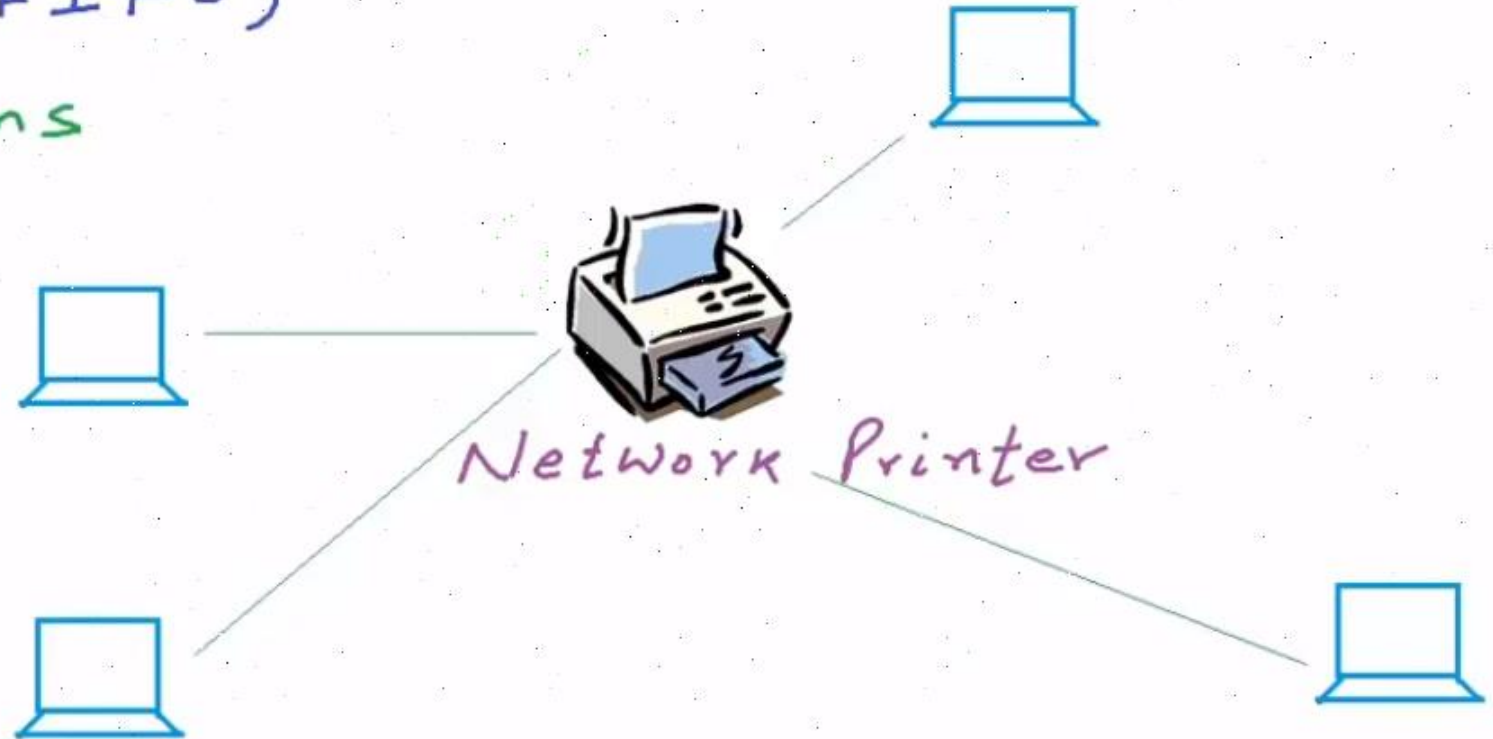
Dequeue()

front() $\Rightarrow$ 5

IsEmpty() $\Rightarrow$ false

Queue Application

Queue ADT (FIFO)
Applications



Queue Implementations

We can implement Queues using:

- 1) Arrays
- 2) Linked Lists

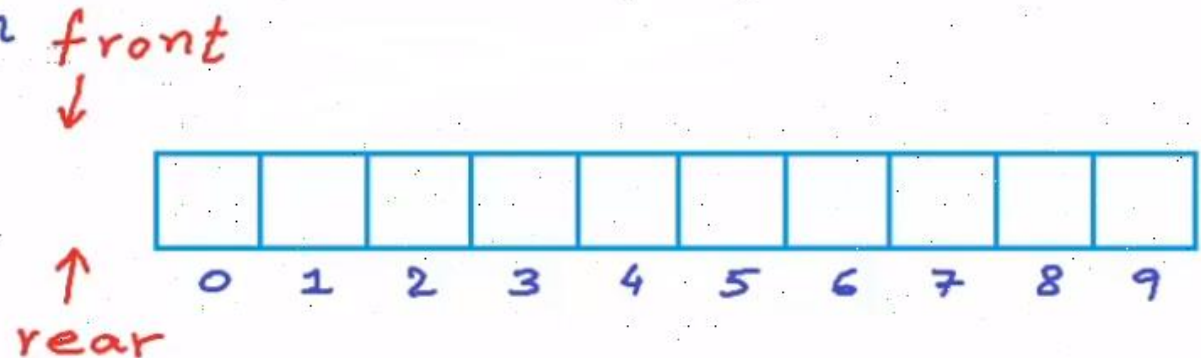
Queue: Array Based Implementation

Array Implementation

```
int A[10]  
front ← -1  
rear ← -1
```

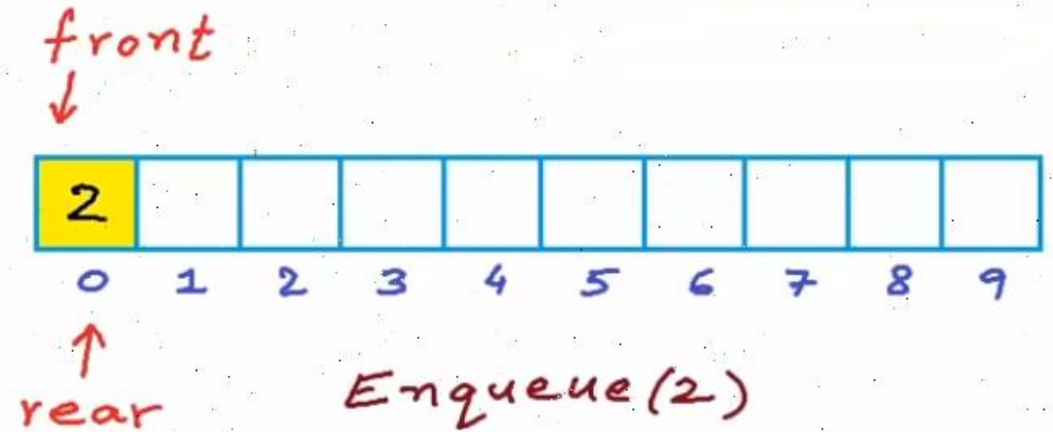
IsEmpty()

```
{ if front == -1 & rear == -1  
  return true  
  else  
  return false  
}
```



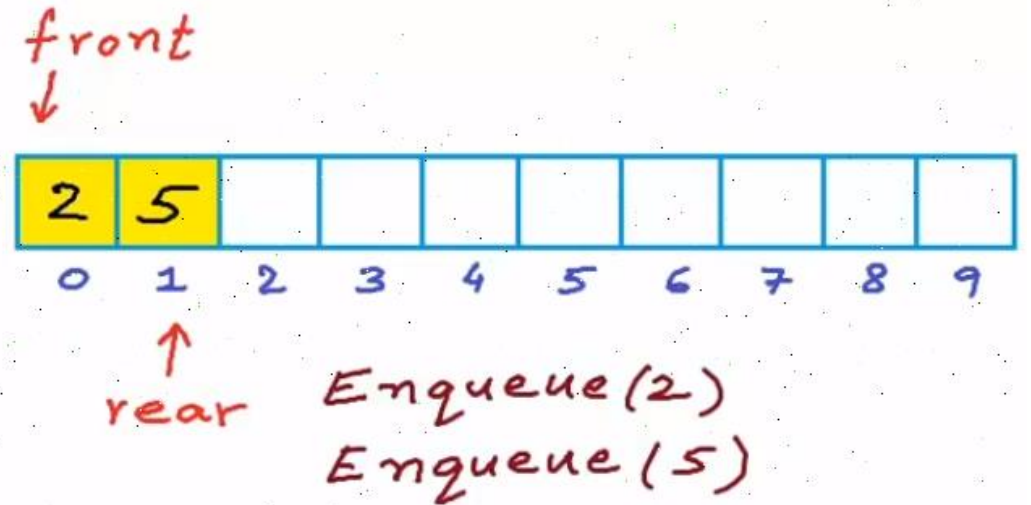
Queue: Array Based Implementation

```
Enqueue(x)
{
  if IsFull()
    return
  else if IsEmpty()
  {
    front ← rear ← 0
  }
  else
  {
    rear ← rear + 1
  }
  ⇒ A[rear] ← x
}
```



Queue: Array Based Implementation

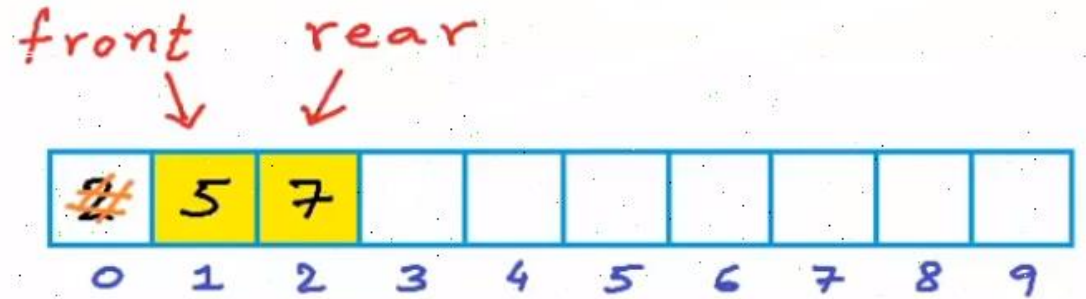
```
Enqueue(x)
{
    if IsFull()
        return
    else if IsEmpty()
    {
        front ← rear ← 0
    }
    else
    {
        rear ← rear + 1
    }
    ⇒ A[rear] ← x
}
```



Queue: Array Based Implementation

```
Dequeue()
```

```
{  
  if IsEmpty()  
    return  
  else if front == rear  
    front ← rear ← -1  
  else  
    front ← front + 1  
}
```



```
Enqueue(2)
```

```
Enqueue(5)
```

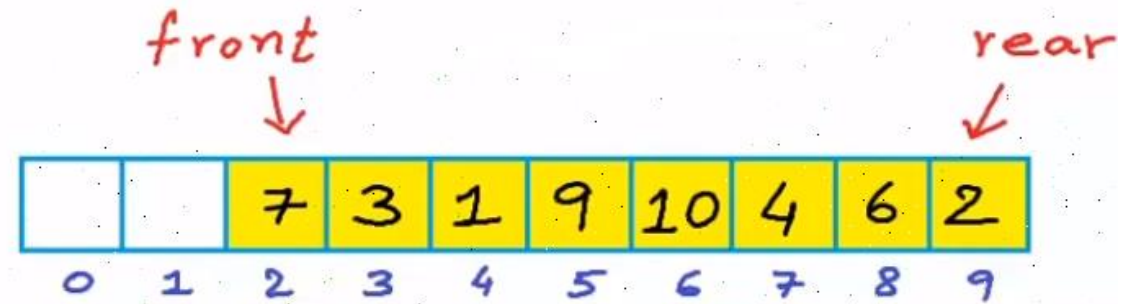
```
Enqueue(7)
```

```
Dequeue()
```


Queue: Array Based Implementation

Dequeue()

```
{  
  if IsEmpty()  
    return  
  else if front == rear  
    front ← rear ← -1  
  else  
    front ← front + 1  
}
```



Enqueue(2)

Enqueue(5)

Enqueue(7)

Dequeue()

Enqueue(3)

Enqueue(1)

Enqueue(9)

Enqueue(10)

Enqueue(4)

Enqueue(6)

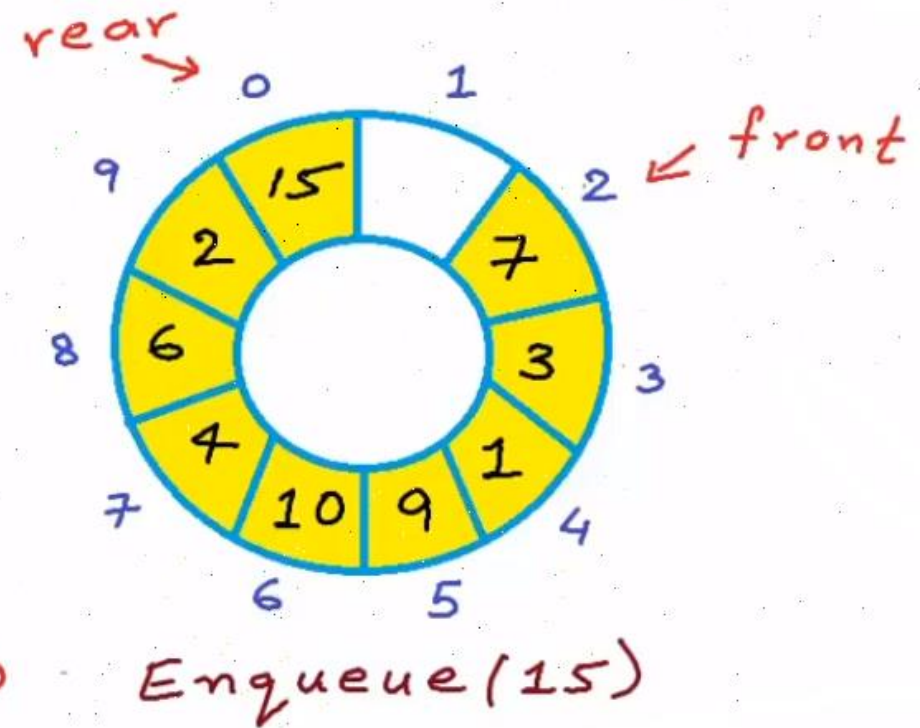
Dequeue()

Enqueue(2)

Circular Queue

```
Enqueue(x)
{
    if (rear+1)%N == front
        return
    else if ISEmpty()
    {
        front ← rear ← 0
    }
    else
    {
        rear ← (rear+1)%N
    }
    A[rear] ← x
}
```

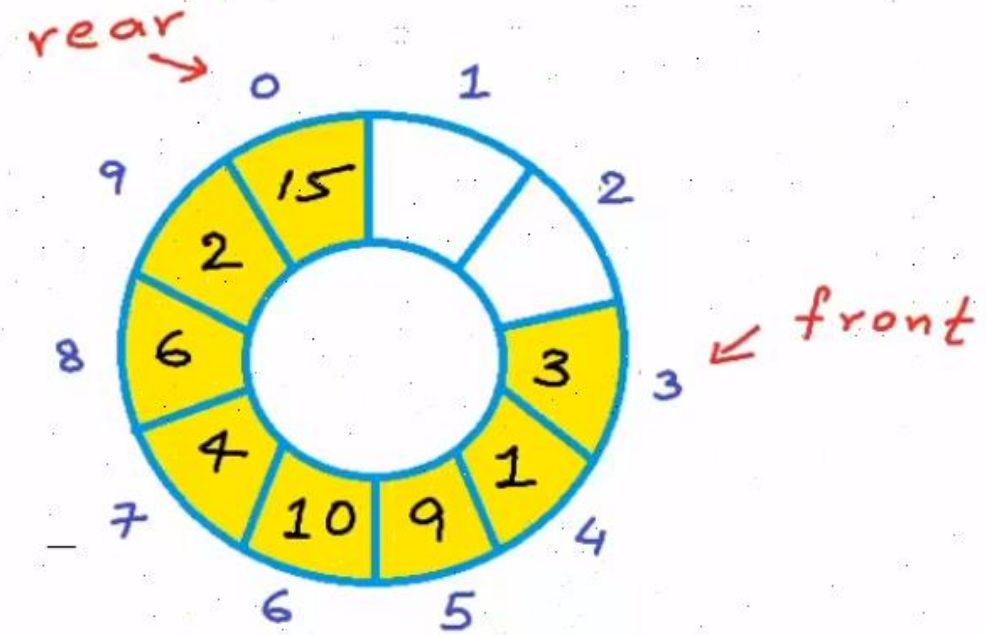
$(9+1) \% 10 = 0$



Circular Queue

```
Dequeue()
```

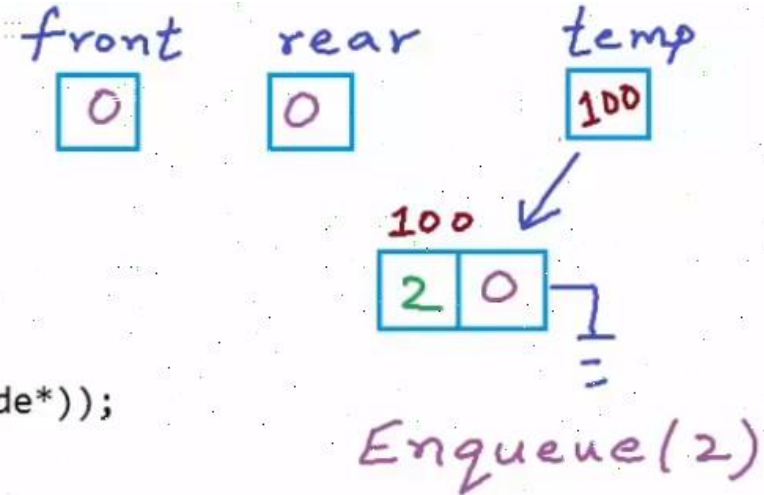
```
{  
  if IsEmpty()  
    return  
  else if front == rear  
    front ← rear ← -1  
  else  
    front ← (front + 1) % N  
}
```



```
Enqueue(15)  
Dequeue()
```

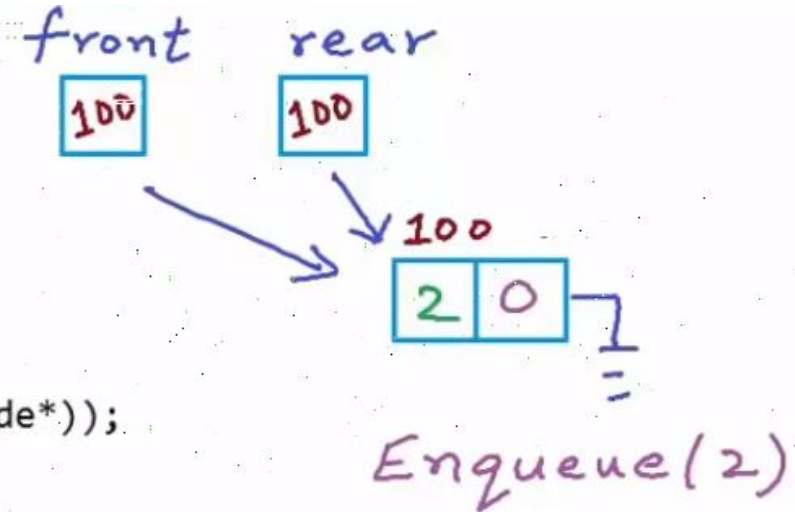
Queue: Linked List Based Implementation

```
struct Node {  
    int data;  
    struct Node* next;  
};  
struct Node* front = NULL;  
struct Node* rear = NULL;  
void Enqueue(int x) {  
    struct Node* temp =  
        (struct Node*)malloc(sizeof(struct Node*));  
    temp->data = x;  
    temp->next = NULL;  
    if(front == NULL && rear == NULL){  
        front = rear = temp;  
        return;  
    }  
    rear->next = temp;  
    rear = temp;  
}
```



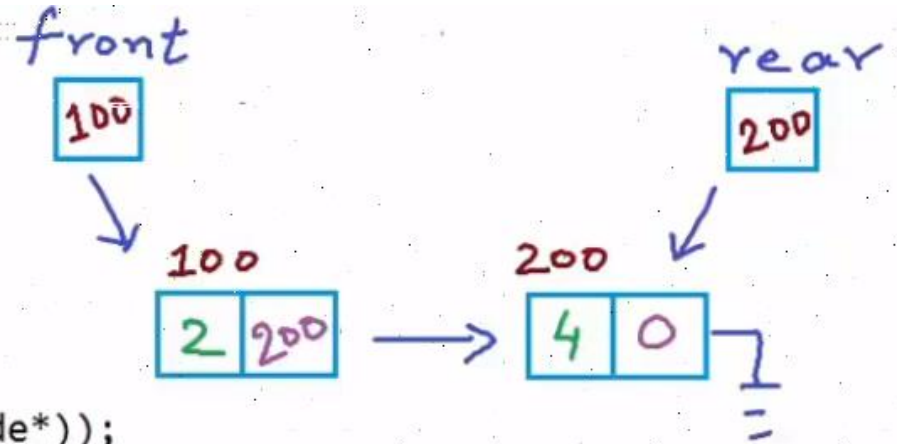
Queue: Linked List Based Implementation

```
struct Node {  
    int data;  
    struct Node* next;  
};  
struct Node* front = NULL;  
struct Node* rear = NULL;  
void Enqueue(int x) {  
    struct Node* temp =  
        (struct Node*)malloc(sizeof(struct Node*));  
    temp->data = x;  
    temp->next = NULL;  
    if(front == NULL && rear == NULL){  
        front = rear = temp;  
        return;  
    }  
    rear->next = temp;  
    rear = temp;  
}
```



Queue: Linked List Based Implementation

```
struct Node {  
    int data;  
    struct Node* next;  
};  
struct Node* front = NULL;  
struct Node* rear = NULL;  
void Enqueue(int x) {  
    struct Node* temp =  
        (struct Node*)malloc(sizeof(struct Node*));  
    temp->data = x;  
    temp->next = NULL;  
    if(front == NULL && rear == NULL){  
        front = rear = temp;  
        return;  
    }  
    rear->next = temp;  
    rear = temp;  
}
```

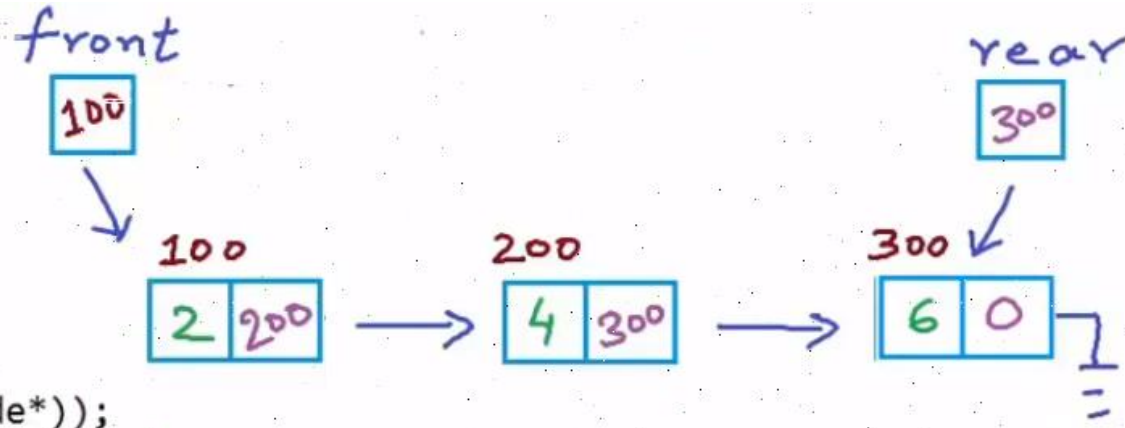


Enqueue(2)

Enqueue(4)

Queue: Linked List Based Implementation

```
struct Node {  
    int data;  
    struct Node* next;  
};  
struct Node* front = NULL;  
struct Node* rear = NULL;  
void Enqueue(int x) {  
    struct Node* temp =  
        (struct Node*)malloc(sizeof(struct Node*));  
    temp->data = x;  
    temp->next = NULL;  
    if(front == NULL && rear == NULL){  
        front = rear = temp;  
        return;  
    }  
    rear->next = temp;  
    rear = temp;  
}
```



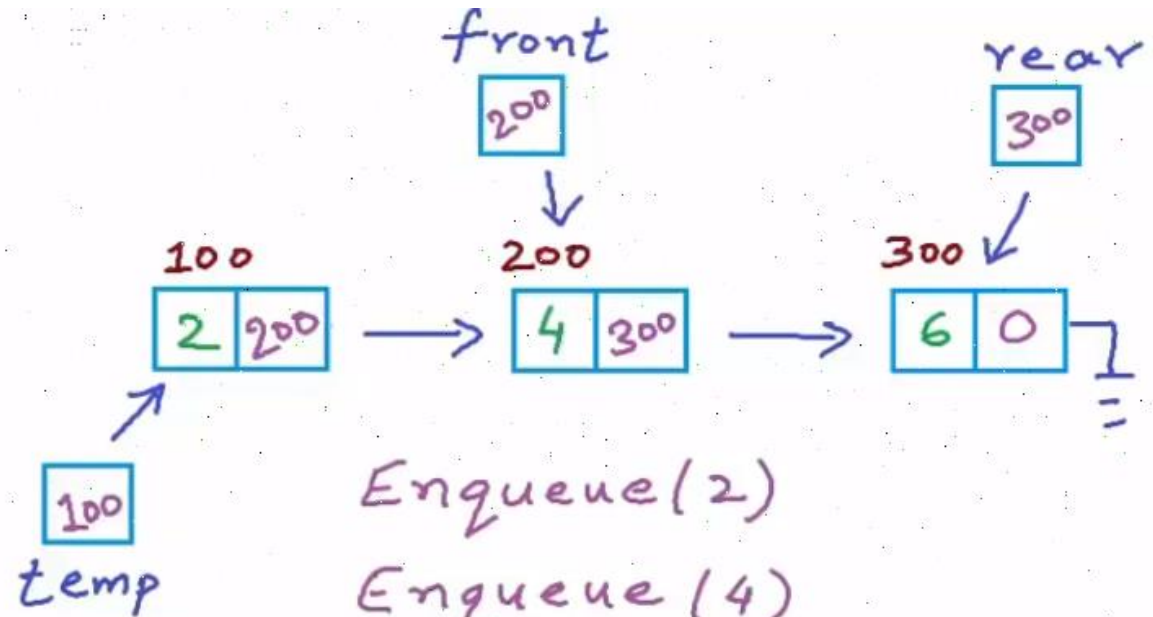
Enqueue(2)

Enqueue(4)

Enqueue(6)

Queue: Linked List Based Implementation

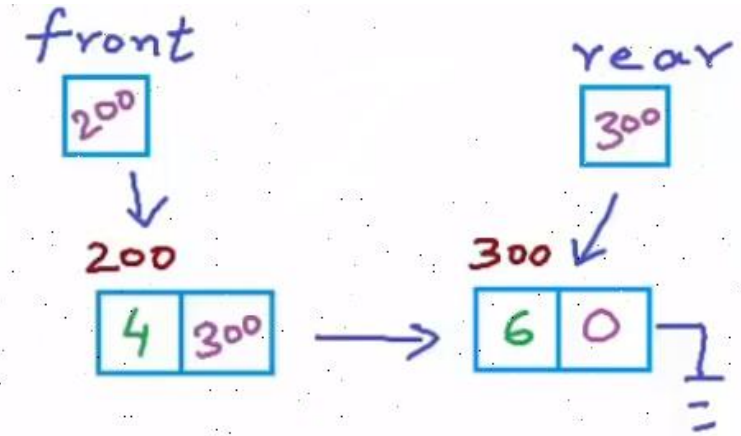
```
void Dequeue() {  
    struct Node* temp = front;  
    if(front == NULL) return;  
    if(front == rear) {  
        front = rear = NULL;  
    }  
    else {  
        front = front->next;  
    }  
    free(temp);  
}
```



Enqueue(2)
Enqueue(4)
Enqueue(6)
Dequeue()

Queue: Linked List Based Implementation

```
void Dequeue() {  
    struct Node* temp = front;  
    if(front == NULL) return;  
    if(front == rear) {  
        front = rear = NULL;  
    }  
    else {  
        front = front->next;  
    }  
    free(temp);  
}
```



Enqueue(2)
Enqueue(4)
Enqueue(6)
Dequeue()